

INTELIGÊNCIA ARTIFICIAL

Regressão Linear

Karina Valdivia Delgado

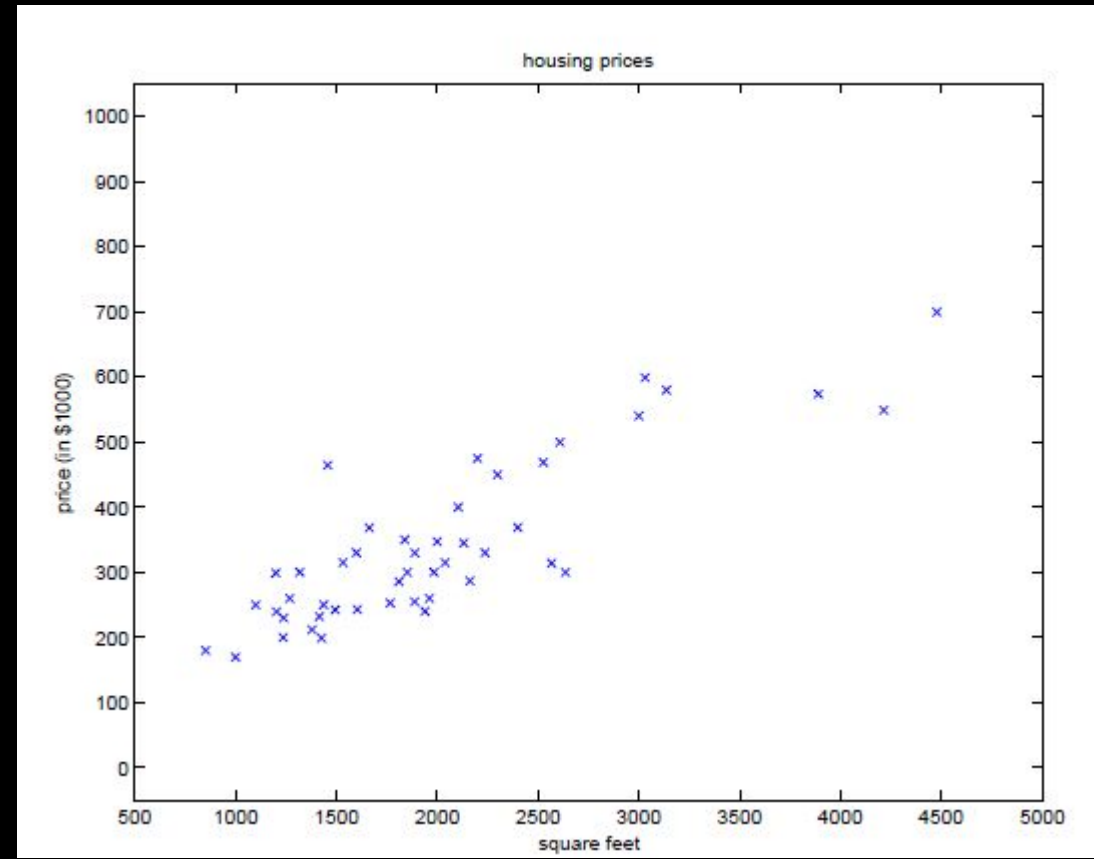
Slides baseados nas notas do prof. Andrew Ng

Temas

- Regressão linear
- Batch gradient descent
- Stochastic gradient descent

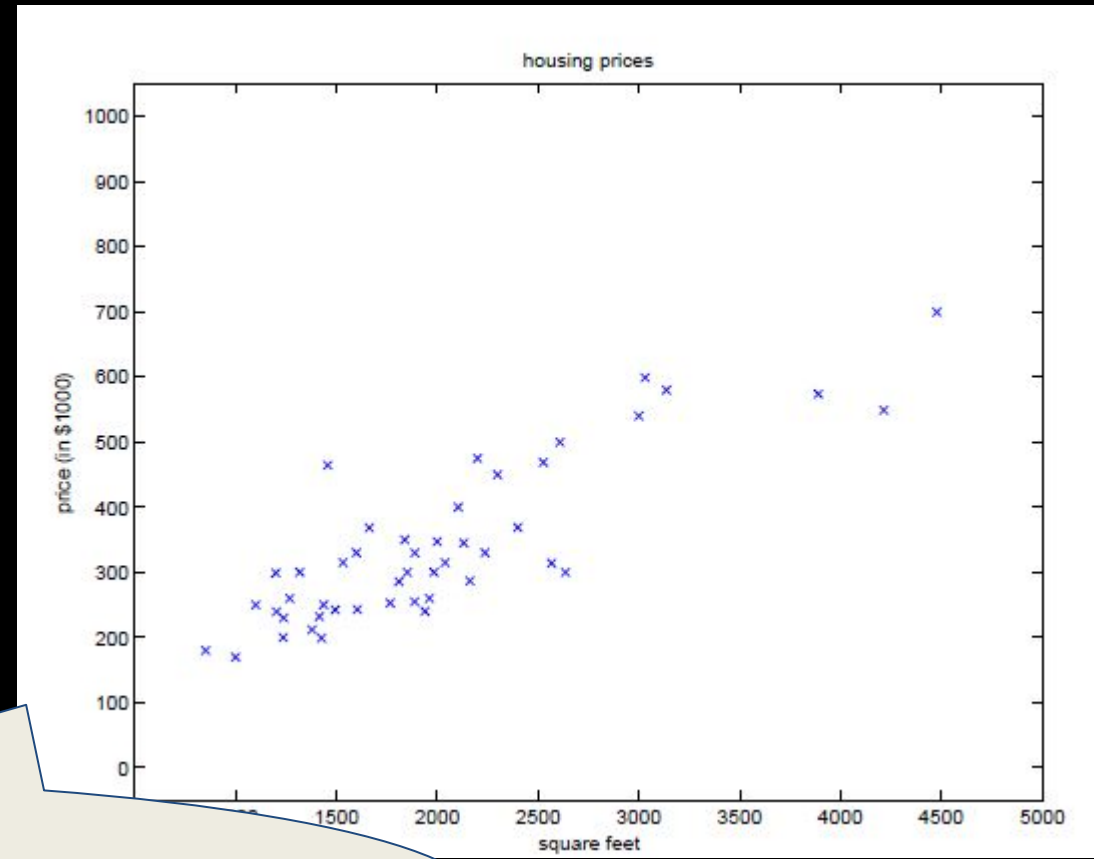
Exemplo de aprendizado supervisionado

Living area (feet ²)	Price (1000\$s)
2104	400
1600	330
2400	369
1416	232
3000	540
⋮	⋮



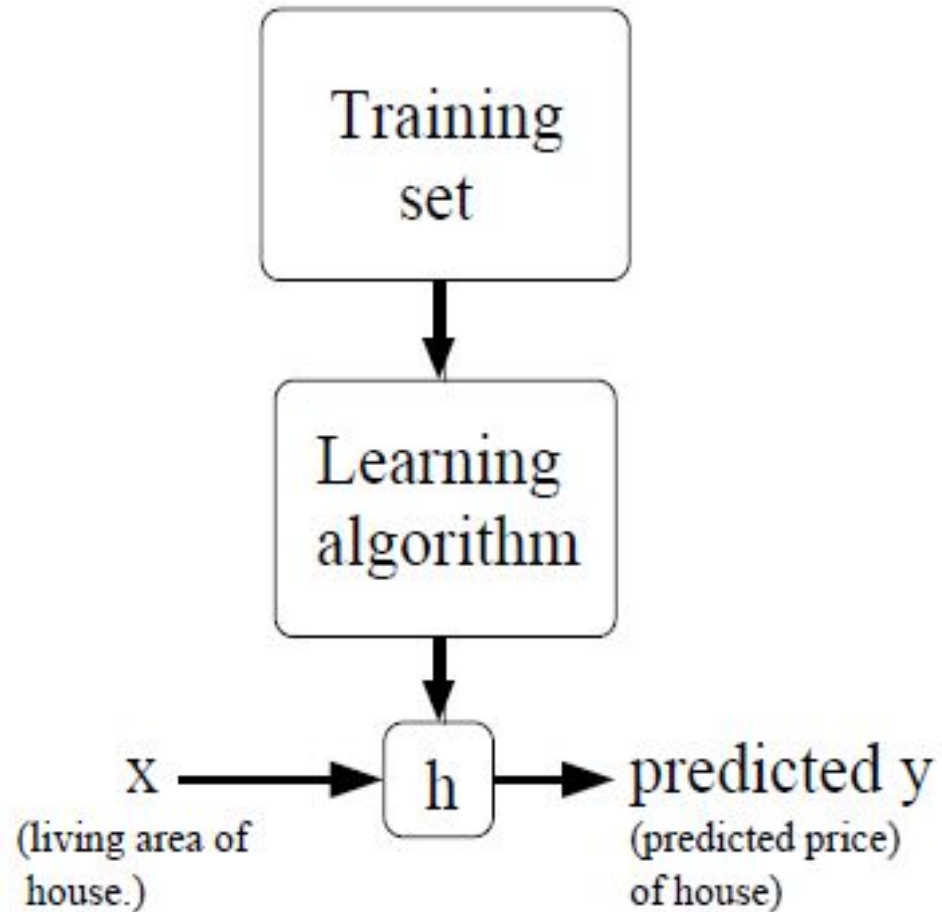
Exemplo de aprendizado supervisionado

Living area (feet ²)	Price (1000\$s)
2104	400
1600	330
2400	369
1416	232
3000	540
⋮	⋮



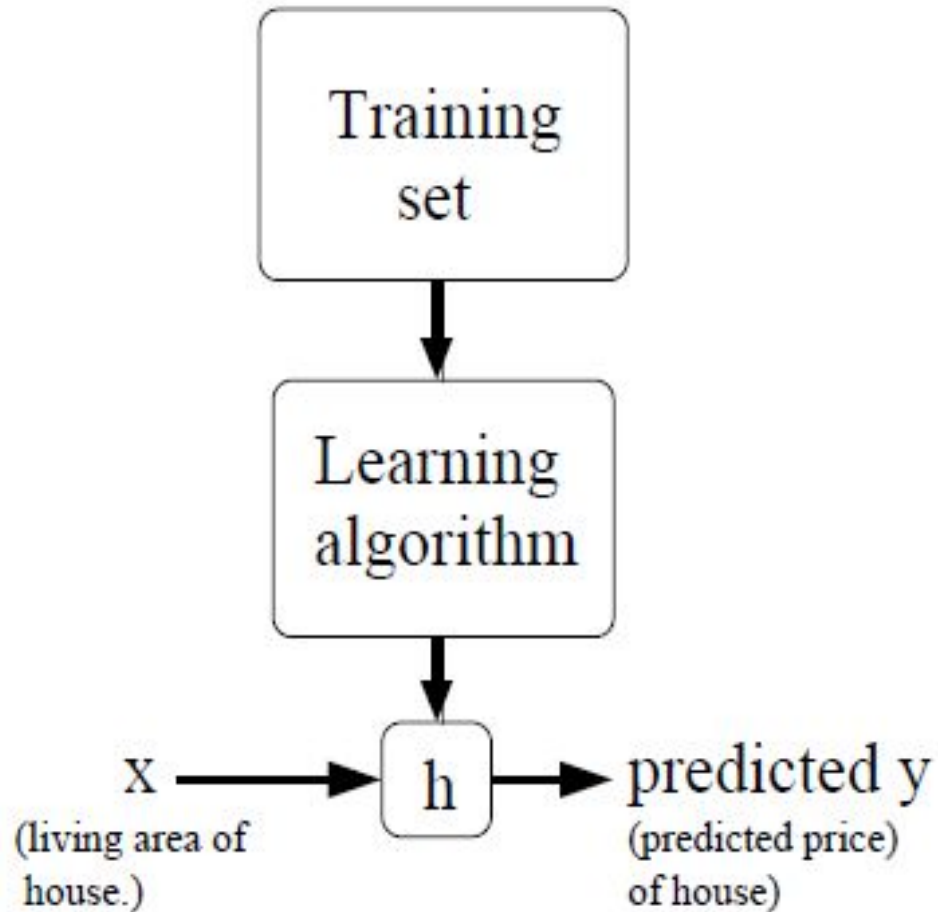
Como prever os preços de outras casas?
O exemplo é um problema de regressão ou
classificação?

Exemplo de aprendizado supervisionado



Como representar h ?

Exemplo de aprendizado supervisionado



Como representar h ?

θ : parâmetros / pesos

n : número de atributos

m : número de exemplos de treinamento

x : atributos

y : rótulo

$(x^{(i)}, y^{(i)})$: i -ésimo exemplo de treinamento

Na regressão linear:

$$h = \theta_0 + \theta_1 x_1$$

Exemplo de aprendizado supervisionado

Living area (feet ²)	#bedrooms	Price (1000\$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
⋮	⋮	⋮

θ : parâmetros / pesos

n : número de atributos

m : número de exemplos de treinamento

x : atributos

y : rótulo

$(x^{(i)}, y^{(i)})$: i -ésimo exemplo de treinamento

Na regressão linear:

$$h = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Exemplo de aprendizado supervisionado

Living area (feet ²)	#bedrooms	Price (1000\$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
⋮	⋮	⋮

θ : parâmetros / pesos

n : número de atributos

m : número de exemplos de treinamento

x : atributos

y : rótulo

$(x^{(i)}, y^{(i)})$: i -ésimo exemplo de treinamento

Na regressão linear:

$$h = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix} \quad x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

Exemplo de aprendizado supervisionado

Living area (feet ²)	#bedrooms	Price (1000\$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
$\vdots$	$\vdots$	$\vdots$

θ : parâmetros / pesos

n : número de atributos

m : número de exemplos de treinamento

x : atributos

y : rótulo

$(x^{(i)}, y^{(i)})$: i -ésimo exemplo de treinamento

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

Na regressão linear:

$$h = \theta_0 + \theta_1 x_1 + \theta_2 x_2 \quad x_0 = 1$$
$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix} \quad x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

Função de custo

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

- Dado um conjunto de dados de treinamento, como aprendemos θ ?

Função de custo

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

- Dado um conjunto de dados de treinamento, como aprendemos θ ?
 - Fazer $h(x)$ perto de y
- Definimos uma função, chamada de função de custo de mínimos quadrados
 - mede quão perto $h(x^{(i)})$ está de seu correspondente $y^{(i)}$

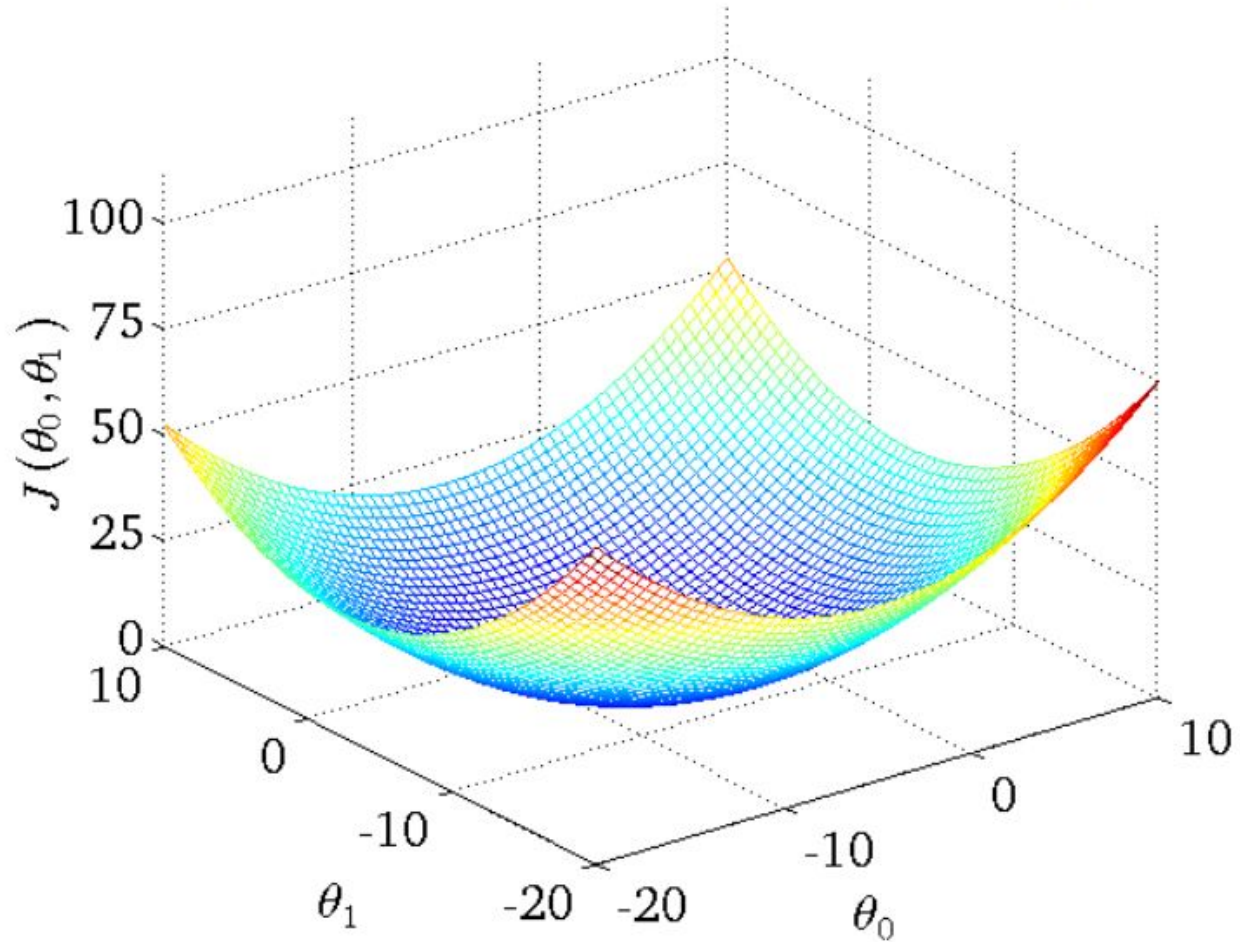
$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- Desejamos minimizar $J(\theta)$

Função de custo

- Considere que temos apenas um atributo x_1
- Ou seja temos dois parâmetros θ_0 θ_1

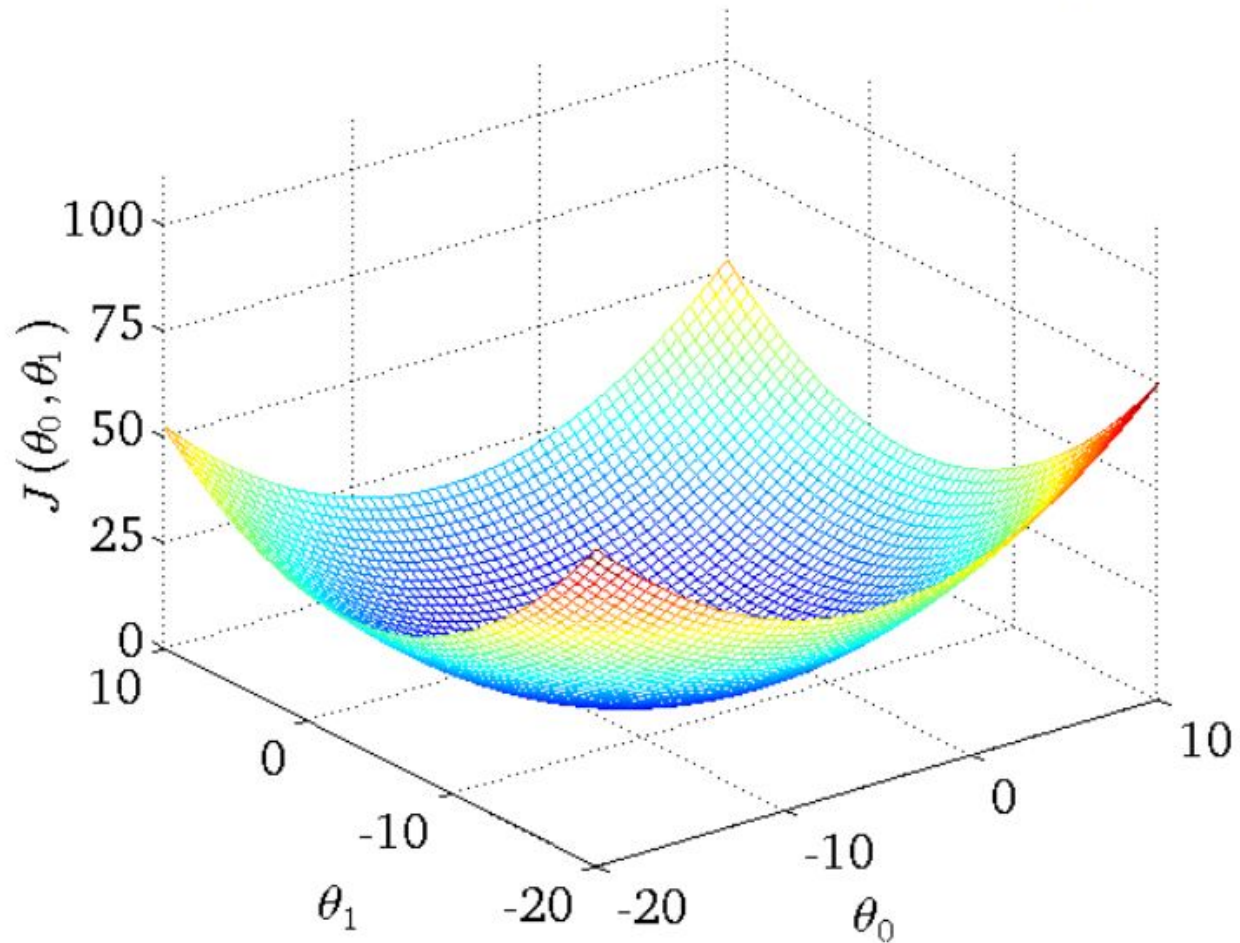
$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



Função de custo

- Considere que temos apenas um atributo x_1
- Ou seja temos dois parâmetros θ_0 θ_1
- Desejamos minimizar $J(\theta)$, precisamos de um algoritmo que permite encontrar o valor desses parâmetros

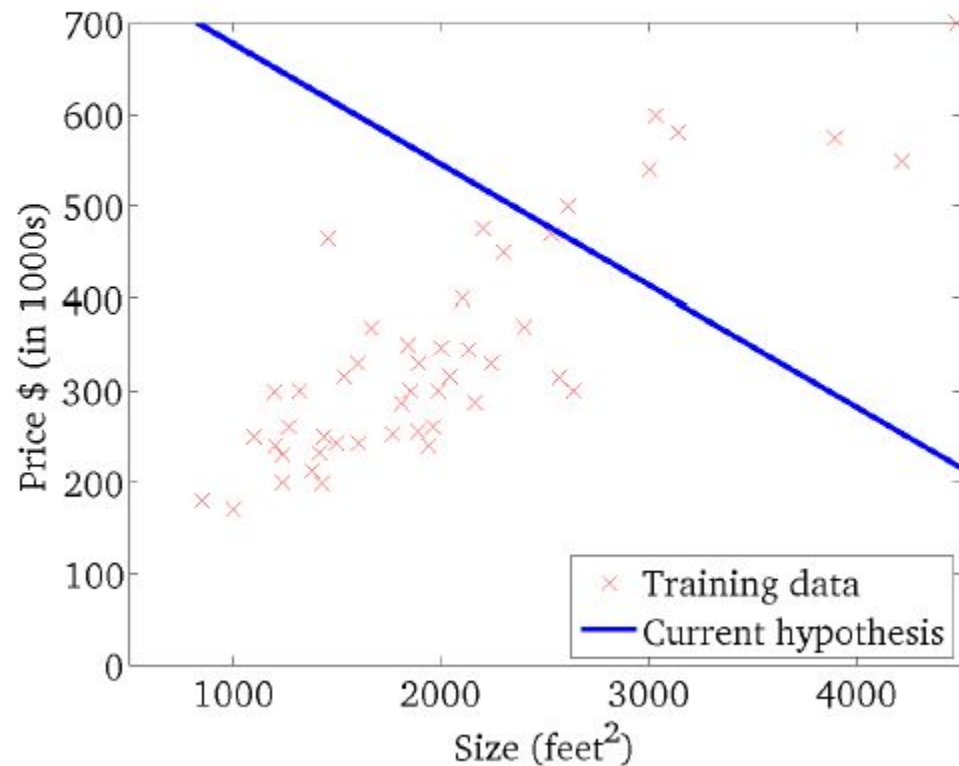
$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



Função de custo

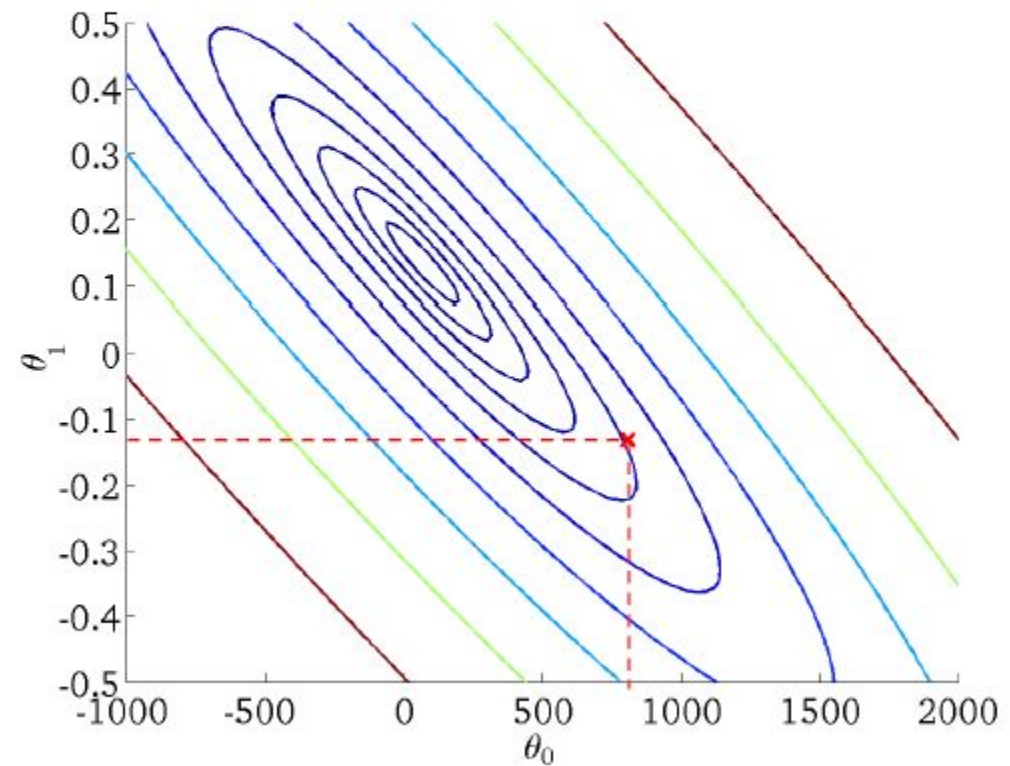
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

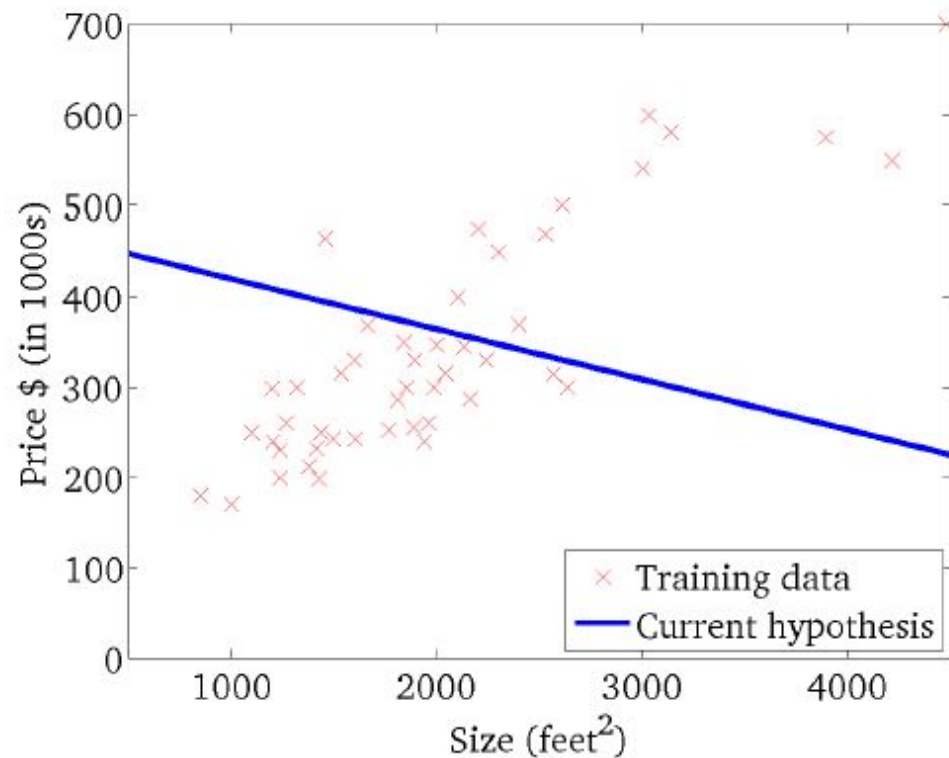
(function of the parameters θ_0, θ_1)



Função de custo

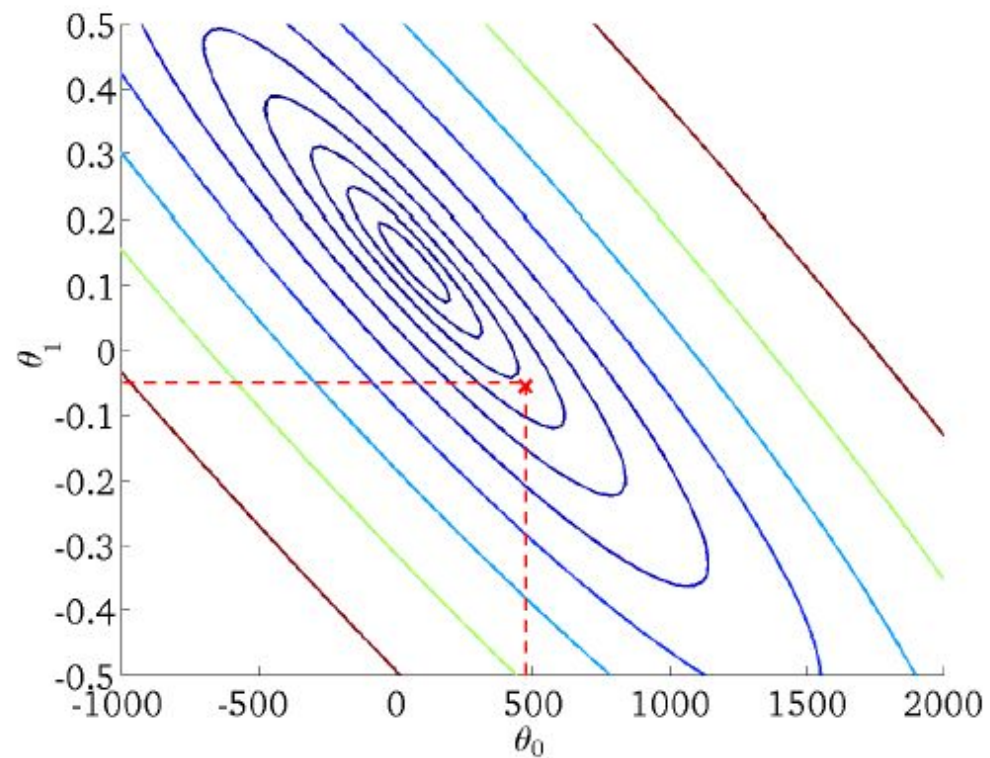
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

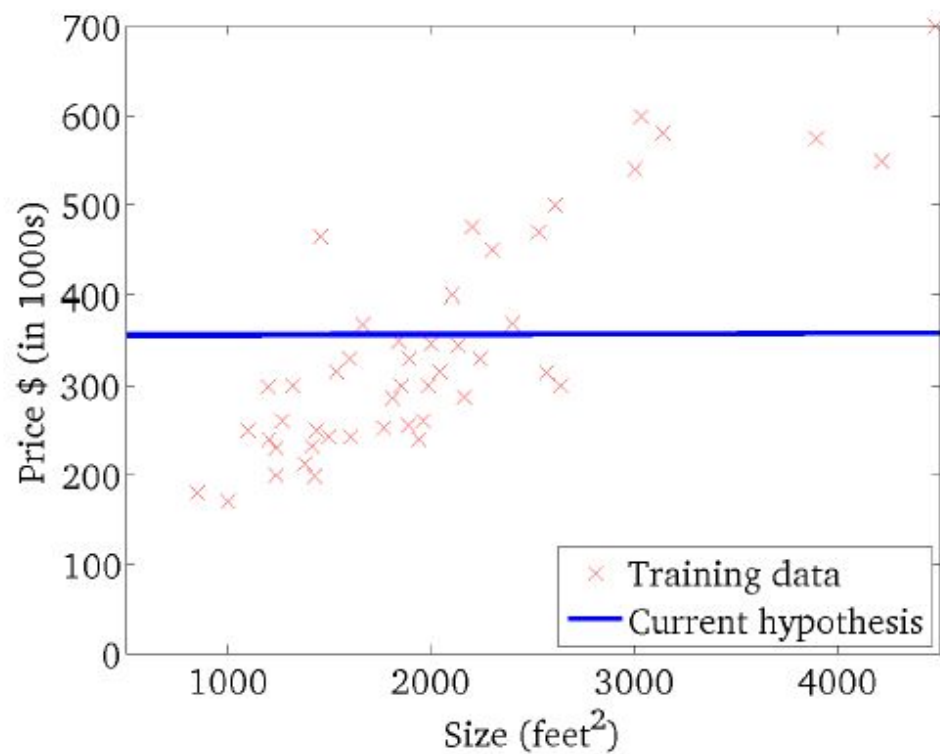
(function of the parameters θ_0, θ_1)



Função de custo

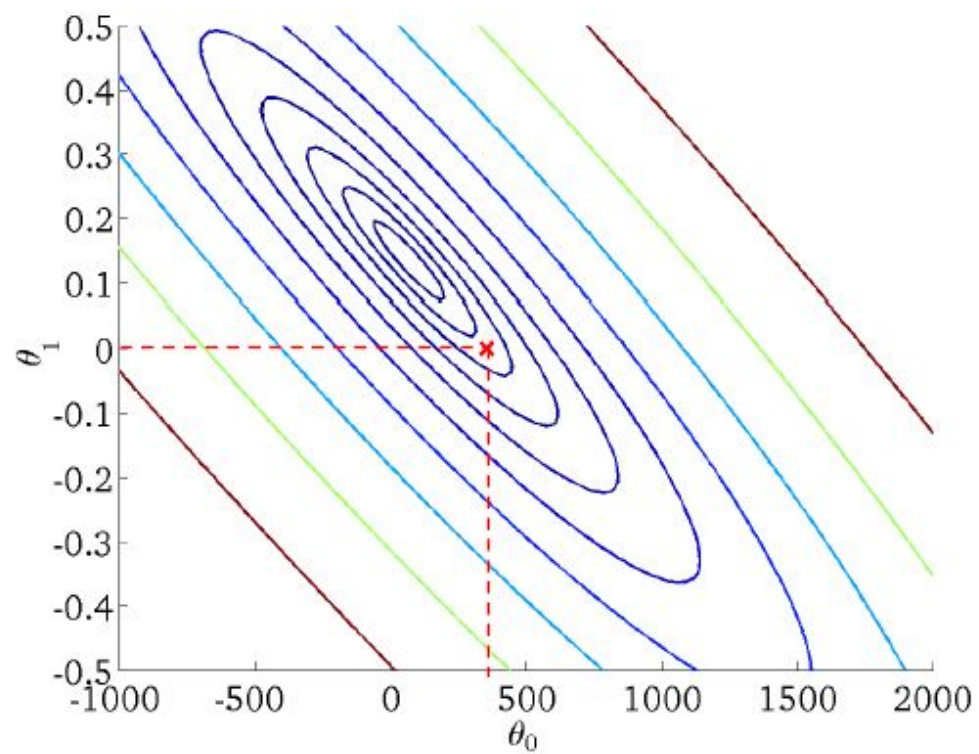
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

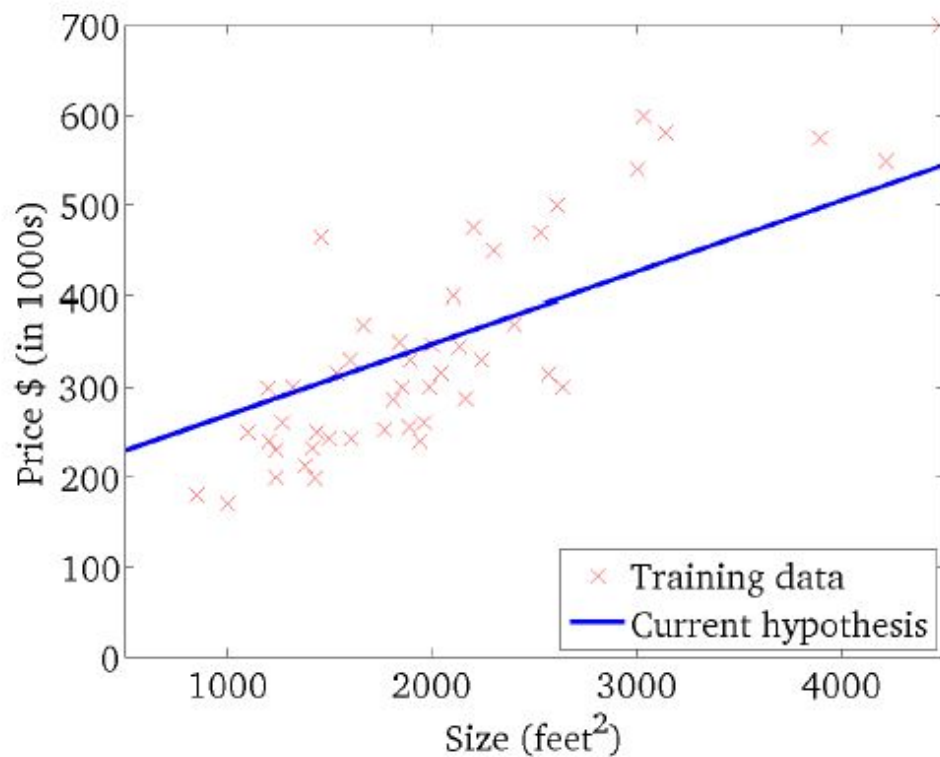
(function of the parameters θ_0, θ_1)



Função de custo

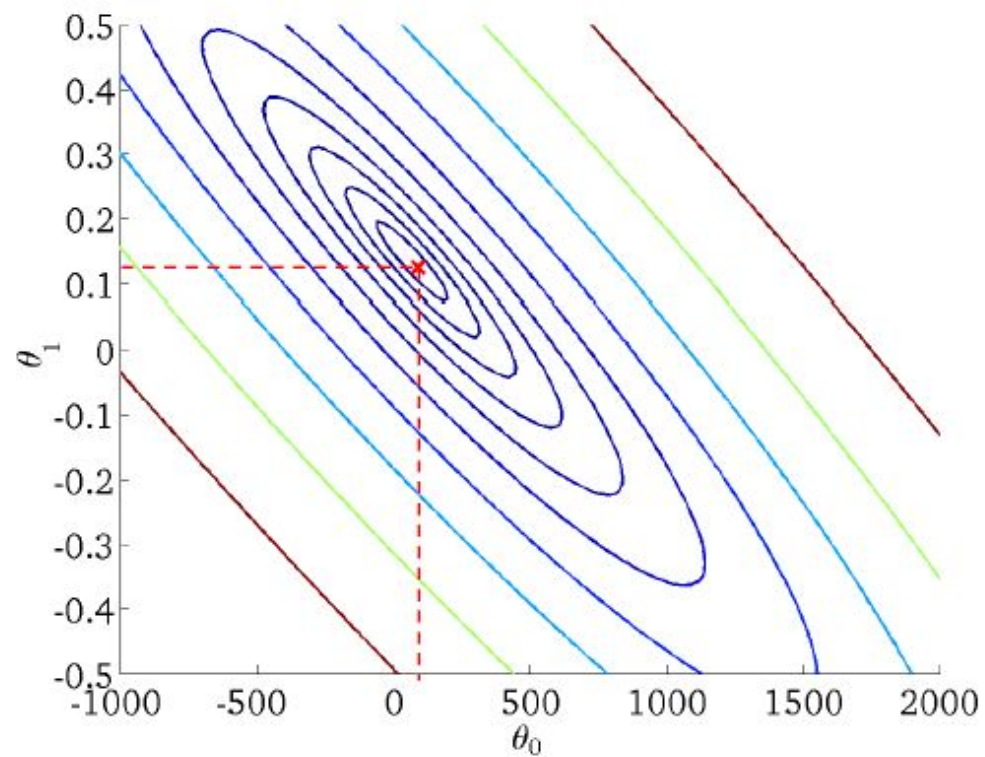
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



Gradient descent

- Algoritmo de busca iterativo que começa com um valor qualquer para θ
- Em cada iteração muda o valor de θ

Gradient descent

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$


simultaneous update
for $j = 0 \dots n$

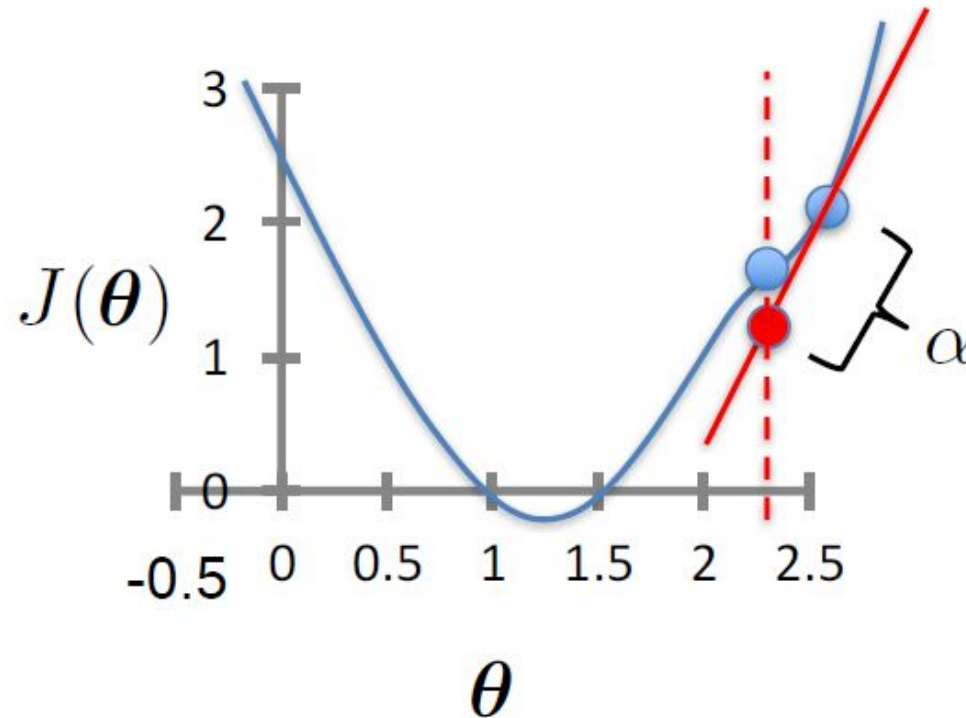
Gradient descent

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

simultaneous update
for $j = 0 \dots n$

learning rate (small)
e.g., $\alpha = 0.05$



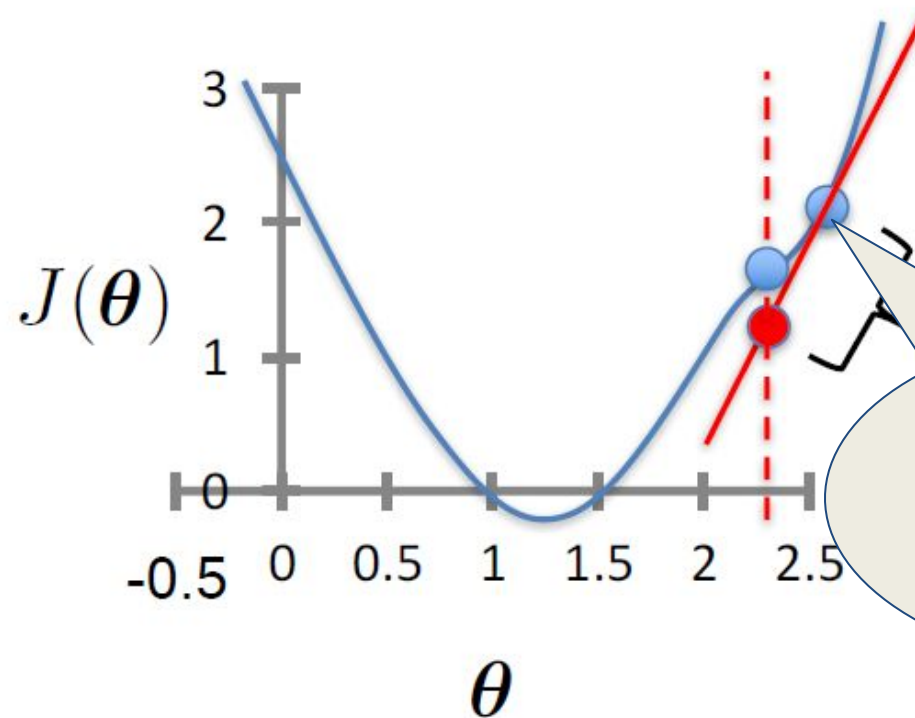
Gradient descent

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

simultaneous update
for $j = 0 \dots n$

learning rate (small)
e.g., $\alpha = 0.05$



Gradient Descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Suponha que apenas temos um exemplo de treinamento

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2$$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Gradient Descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Suponha que apenas temos um exemplo de treinamento

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \end{aligned}$$

Gradient Descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Suponha que apenas temos um exemplo de treinamento

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \end{aligned}$$

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

Gradient Descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Suponha que apenas temos um exemplo de treinamento

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^n \theta_i x_i - y \right) \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

Gradient Descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Suponha que apenas temos um exemplo de treinamento

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} (h_{\theta}(x) - y)^2 \\ &= 2 \cdot \frac{1}{2} (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^n \theta_i x_i - y \right) \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

Se encontramos um exemplo de treinamento no qual nossa previsão quase corresponde ao valor real de y , então existe pouca necessidade de alterar os parâmetros

Para considerar os m exemplos de treinamento, temos as seguintes opções:

1. Batch gradient descent
2. Stochastic gradient descent

(Batch) gradient descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Considerando que temos m exemplos

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (\text{for every } j).$$

}

(Batch) gradient descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Considerando que temos m exemplos

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (\text{for every } j).$$

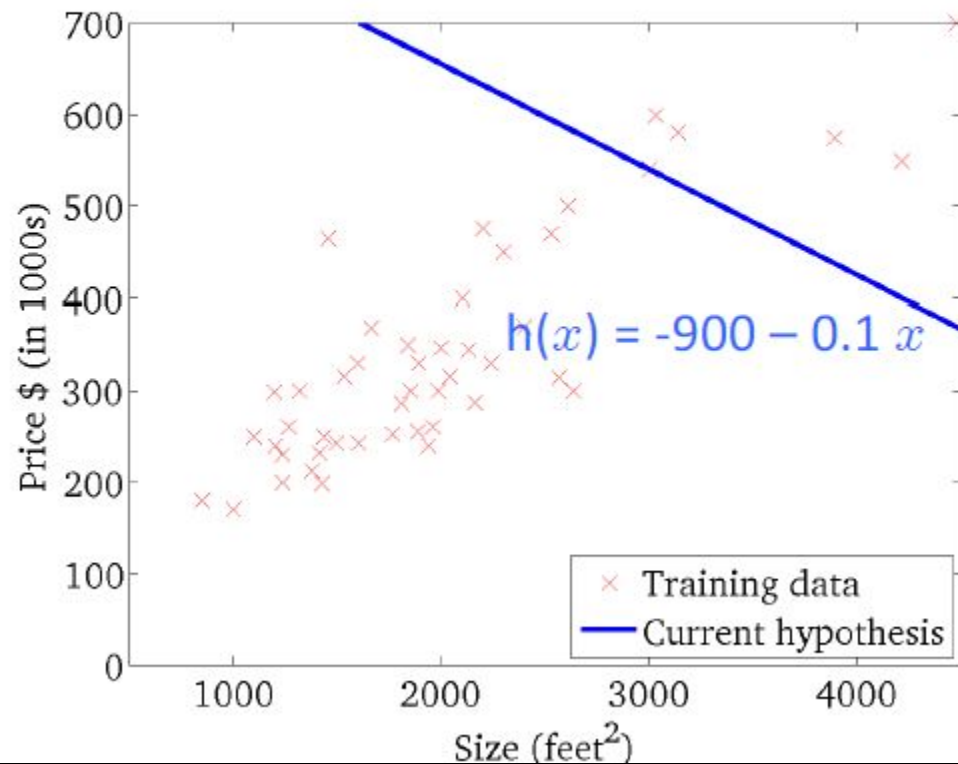
}

Esse método atualiza os pesos depois de ver o conjunto completo de exemplos de treinamento. e é chamado de batch gradient descent.

Batch gradient descent

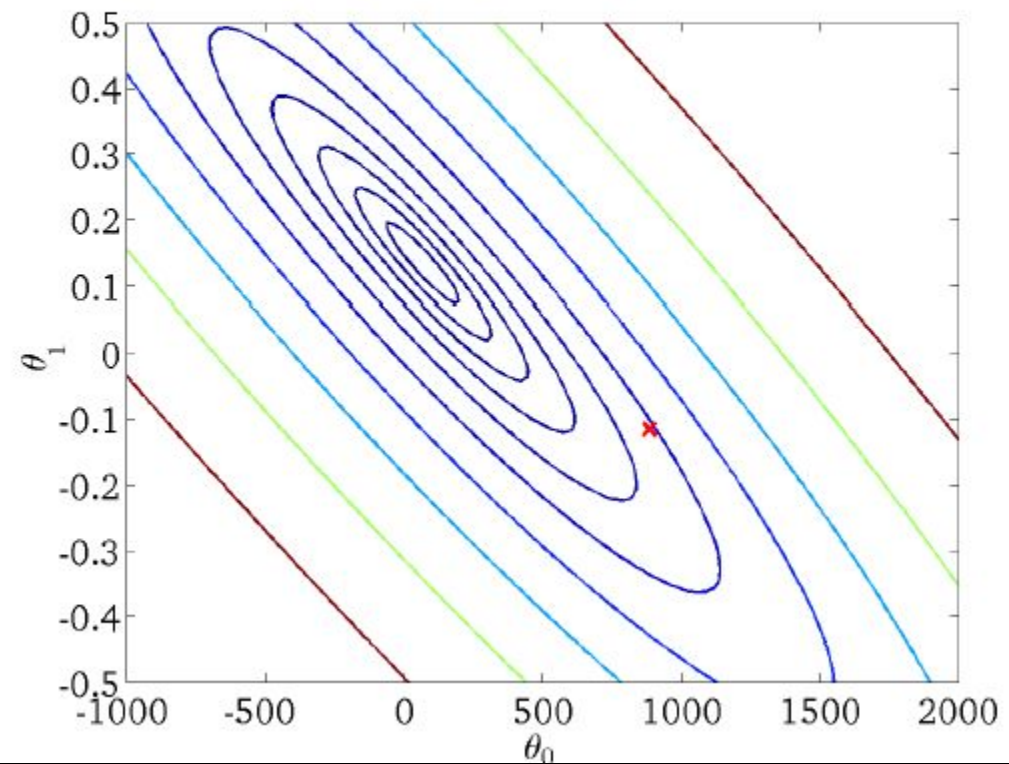
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

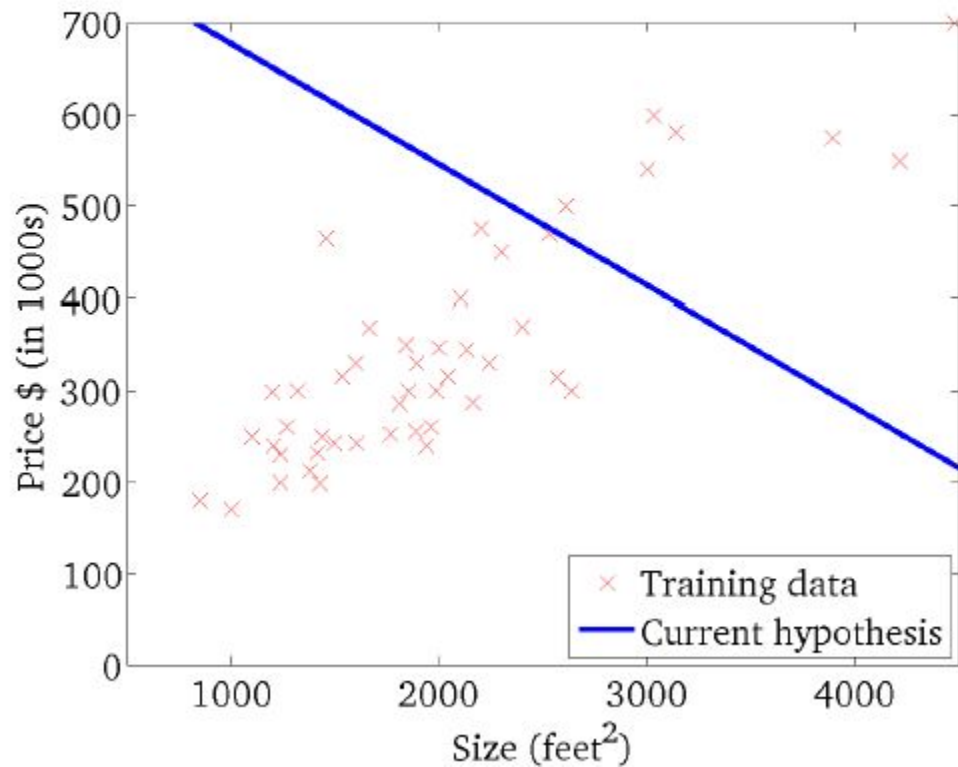
(function of the parameters θ_0, θ_1)



Batch gradient descent

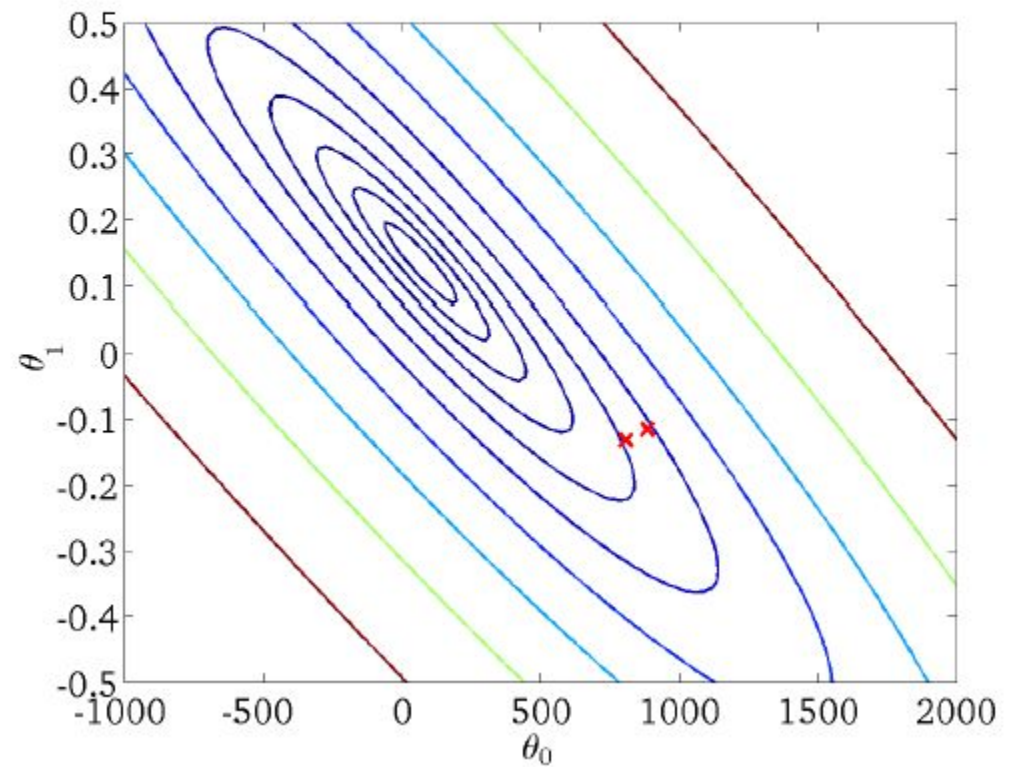
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

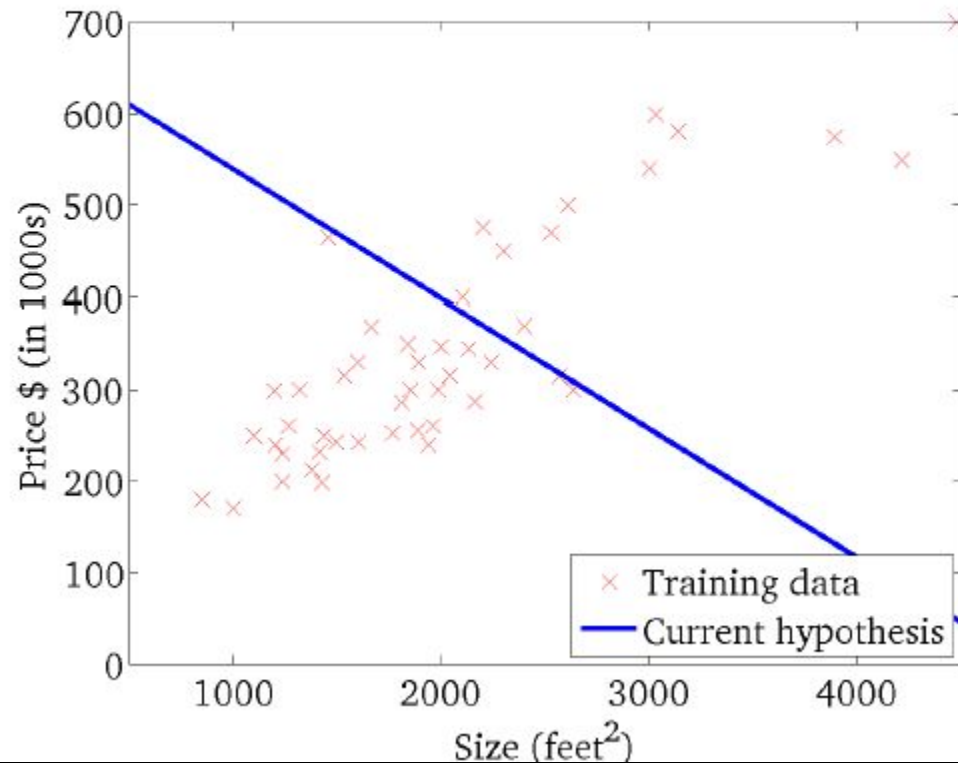
(function of the parameters θ_0, θ_1)



Batch gradient descent

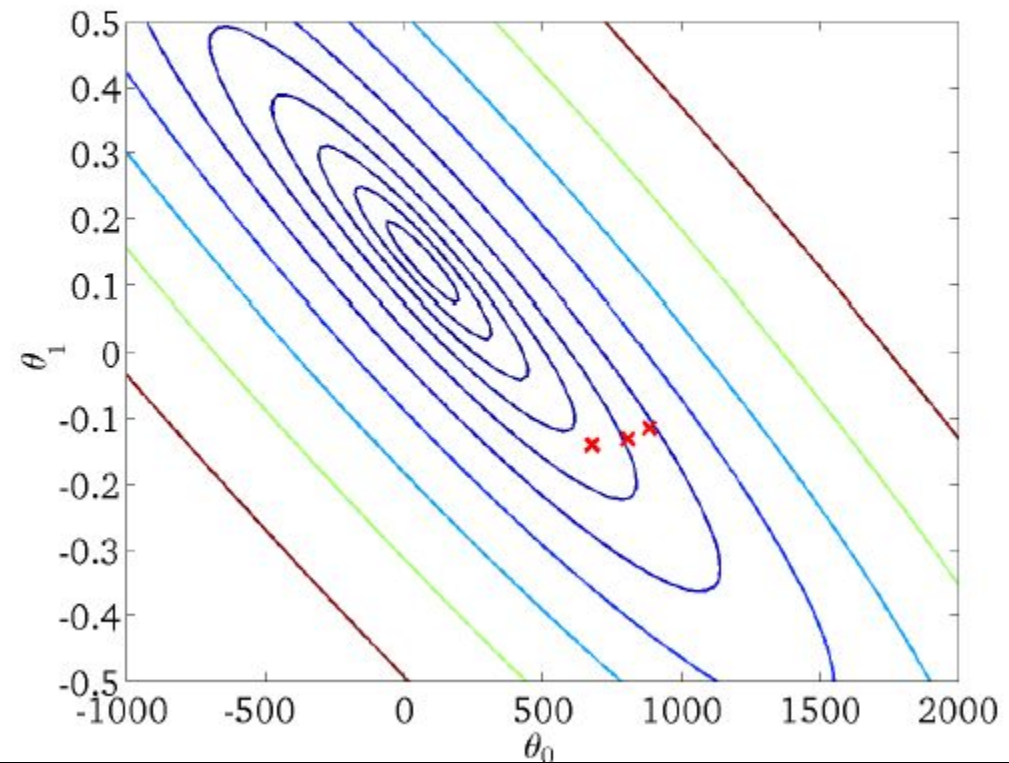
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

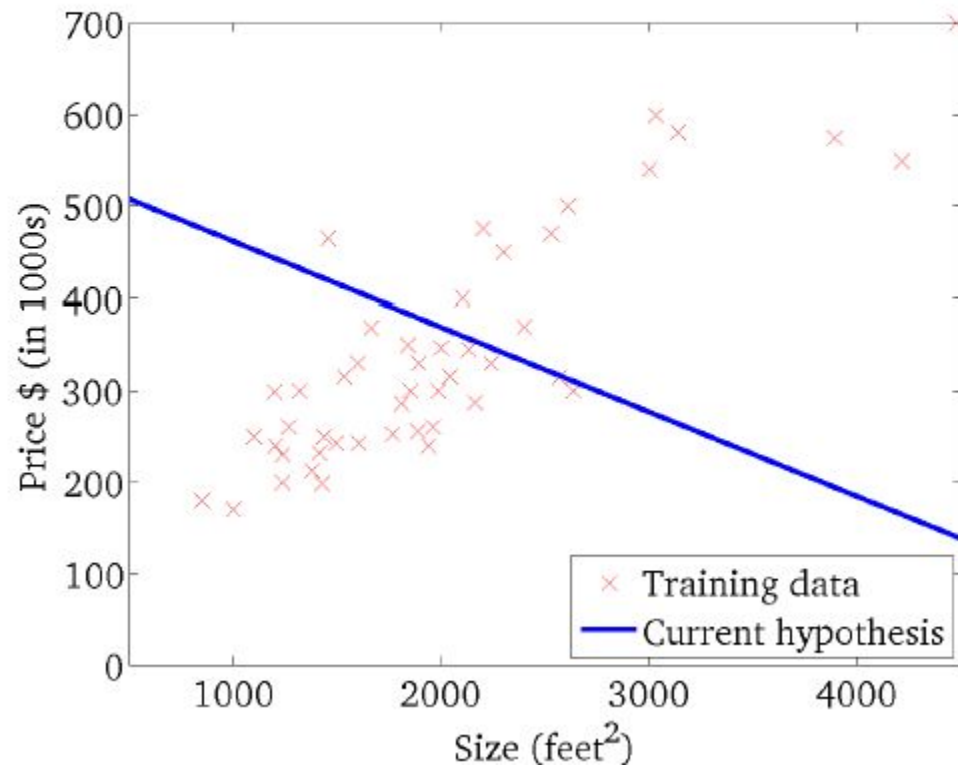
(function of the parameters θ_0, θ_1)



Batch gradient descent

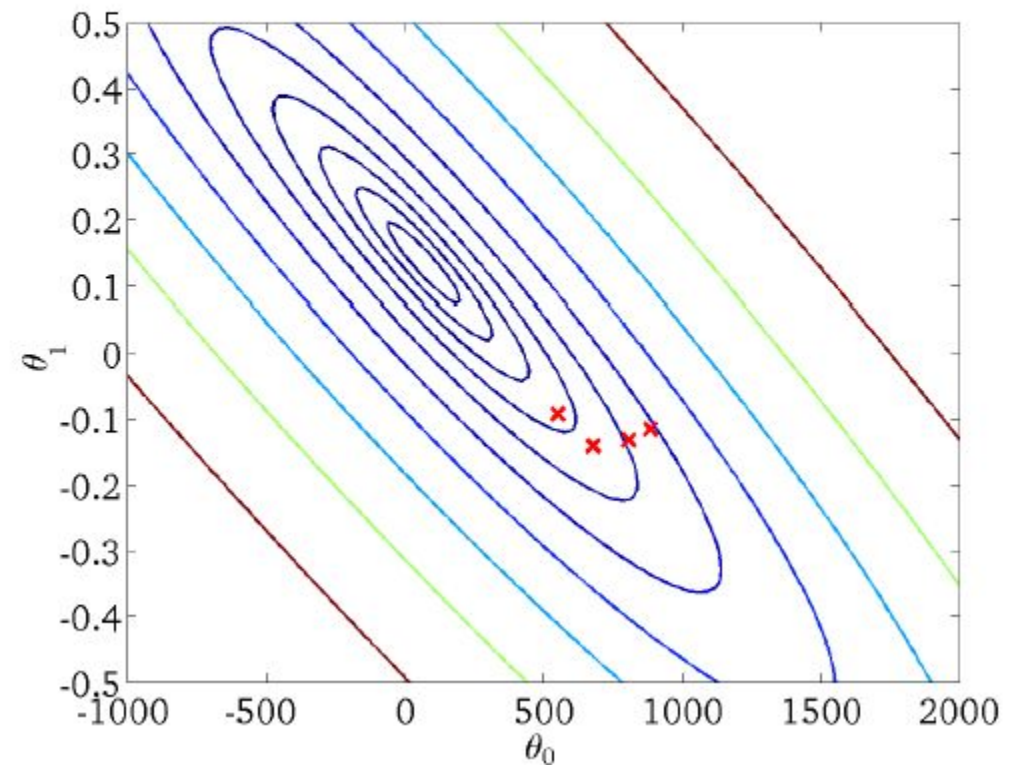
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

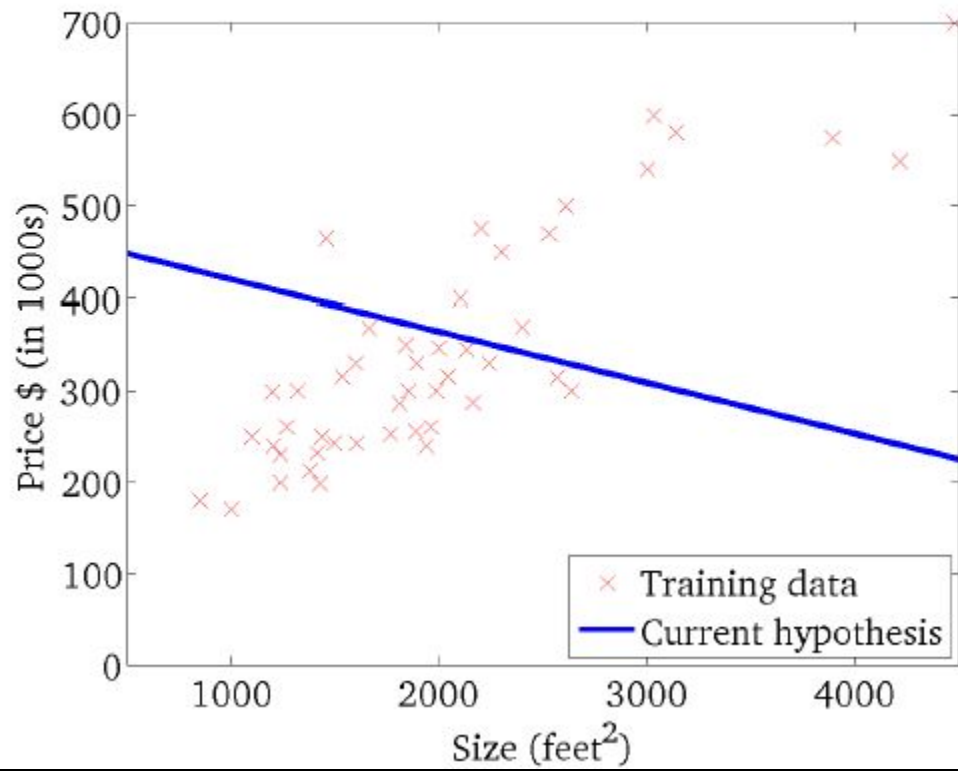
(function of the parameters θ_0, θ_1)



Batch gradient descent

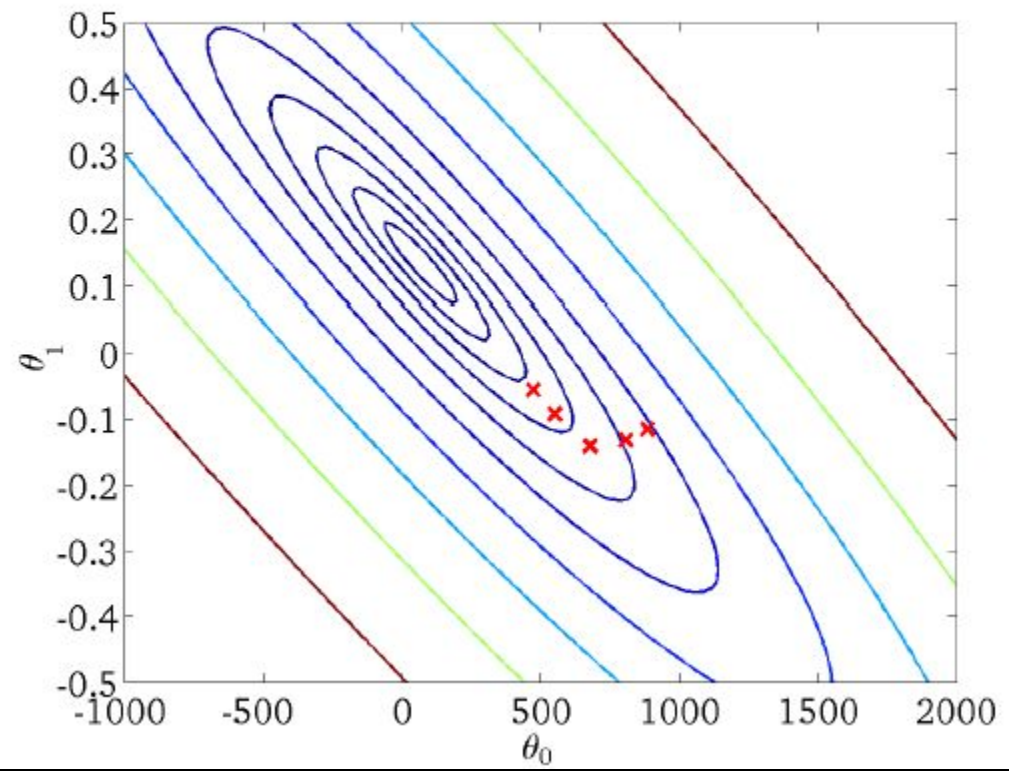
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

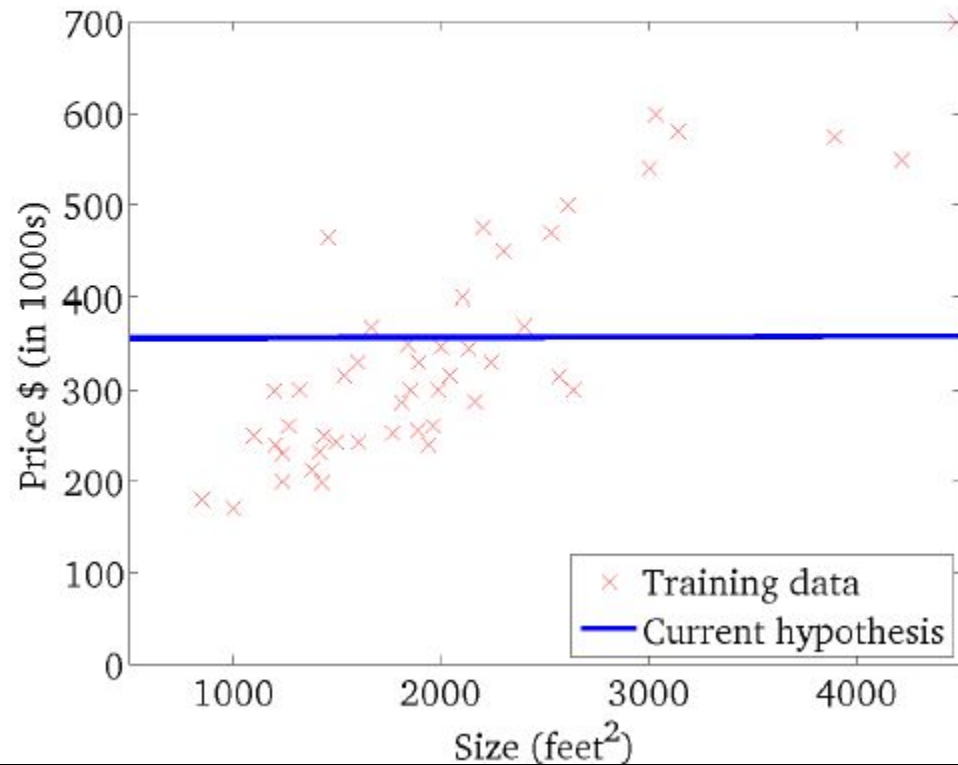
(function of the parameters θ_0, θ_1)



Batch gradient descent

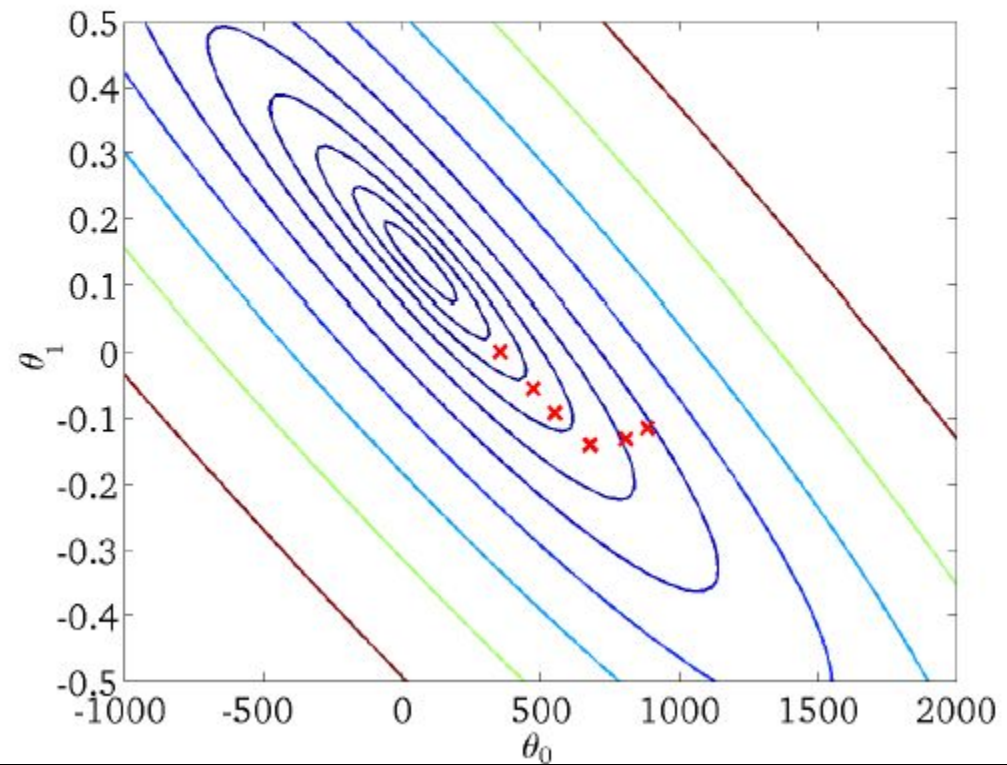
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

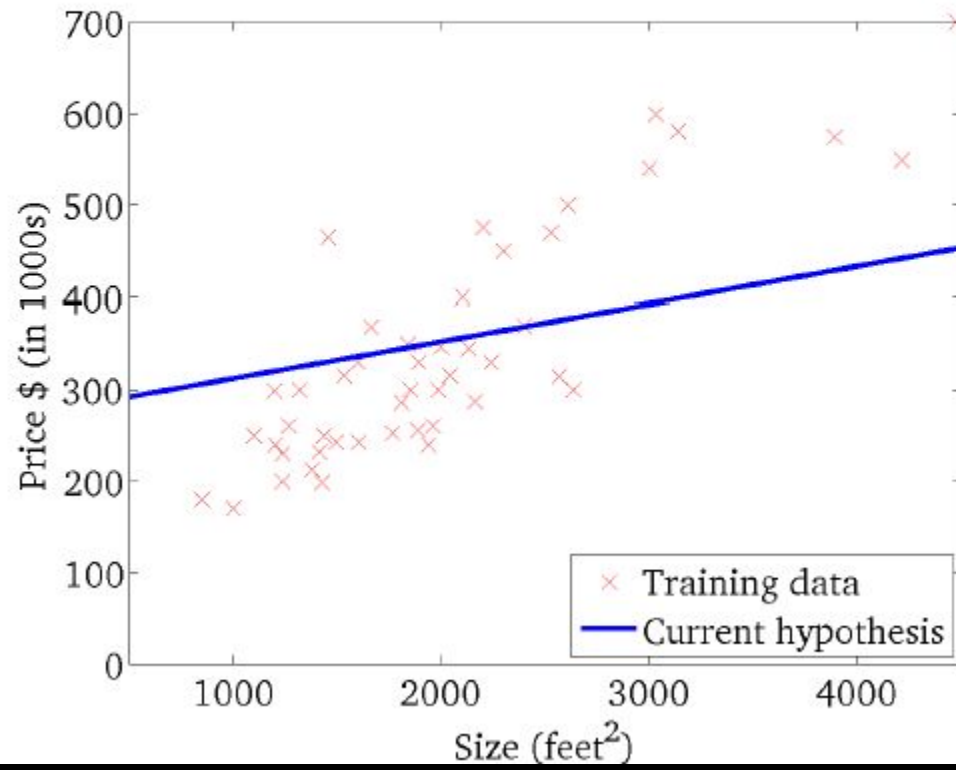
(function of the parameters θ_0, θ_1)



Batch gradient descent

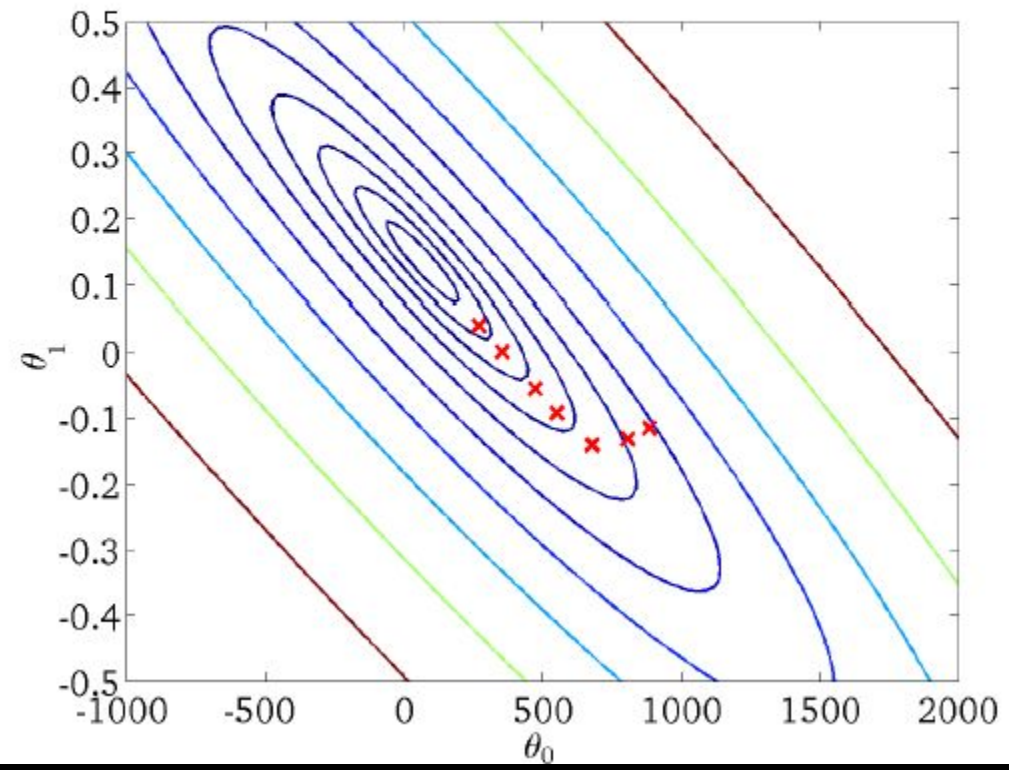
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

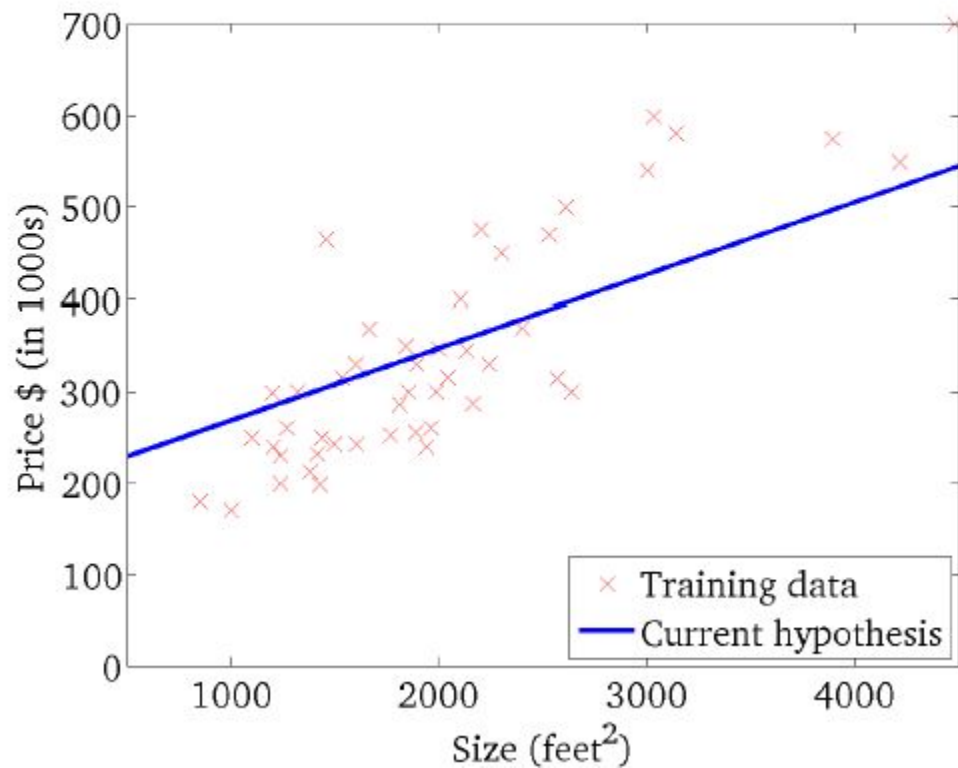
(function of the parameters θ_0, θ_1)



Batch gradient descent

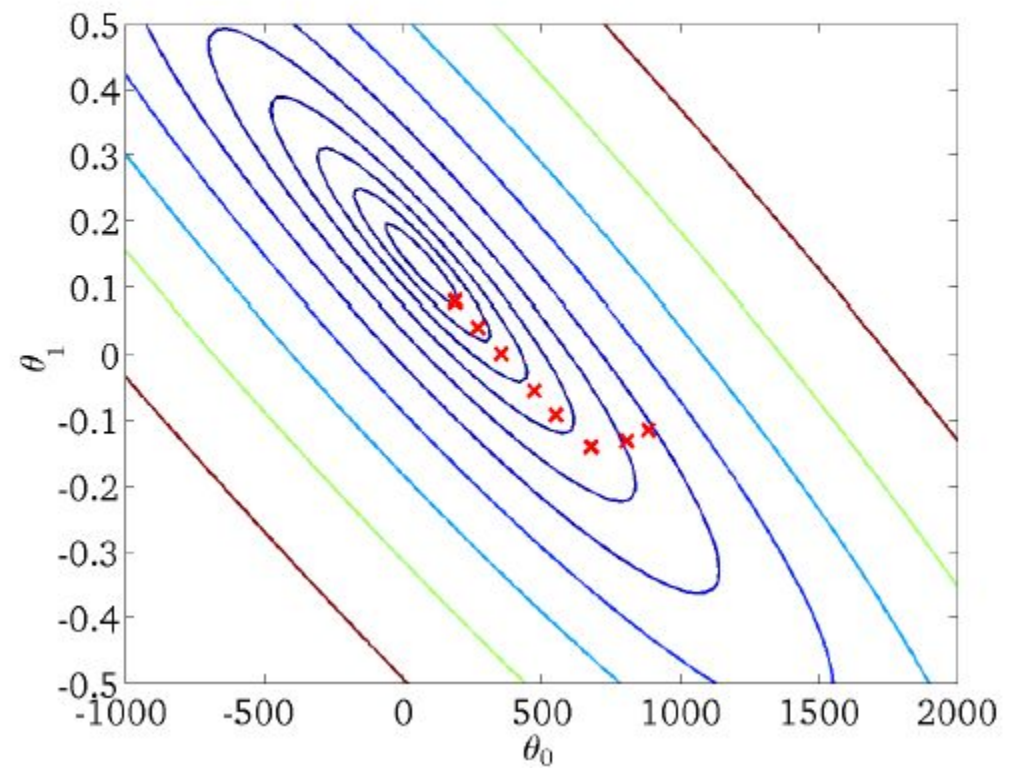
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

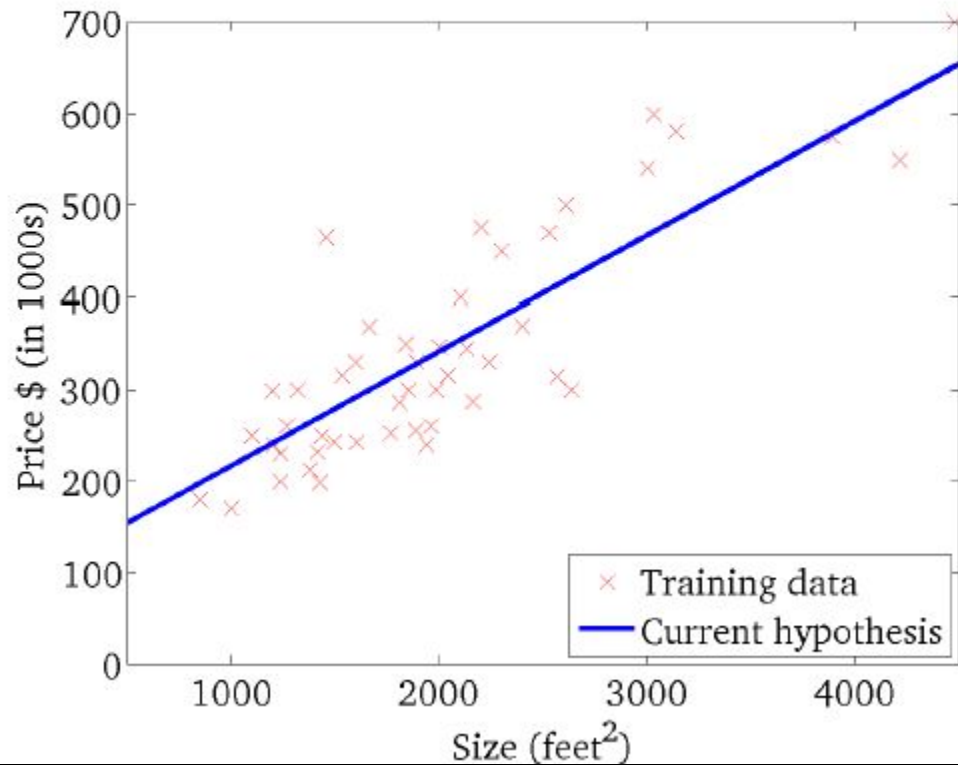
(function of the parameters θ_0, θ_1)



Batch gradient descent

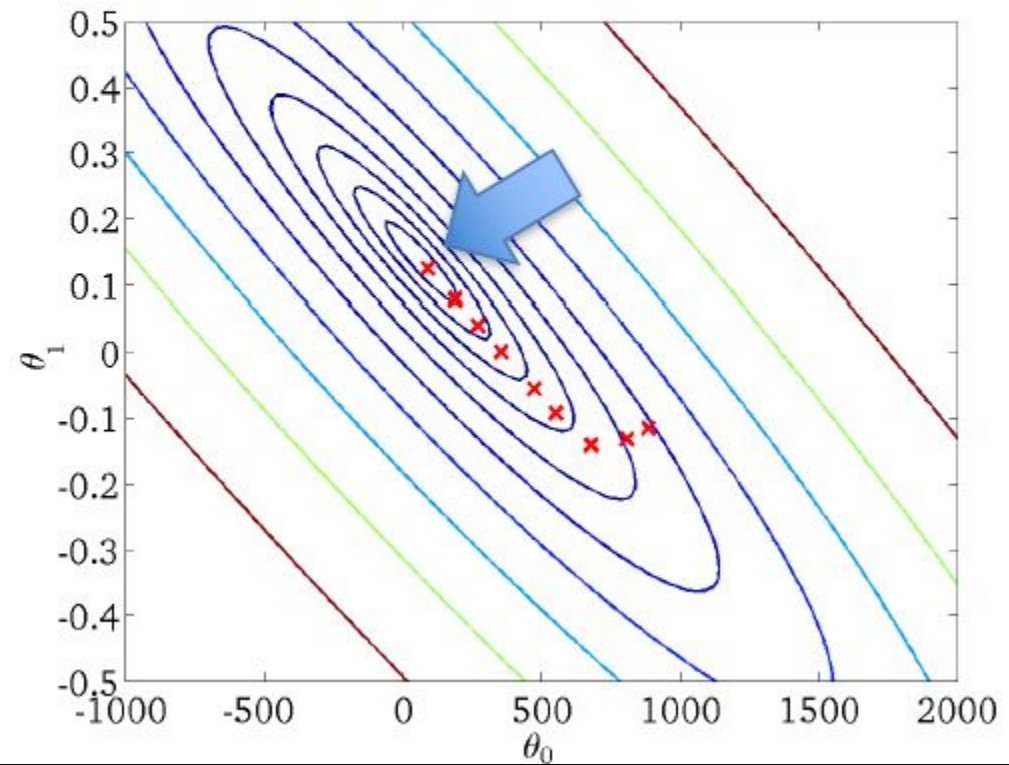
$$h_{\theta}(x)$$

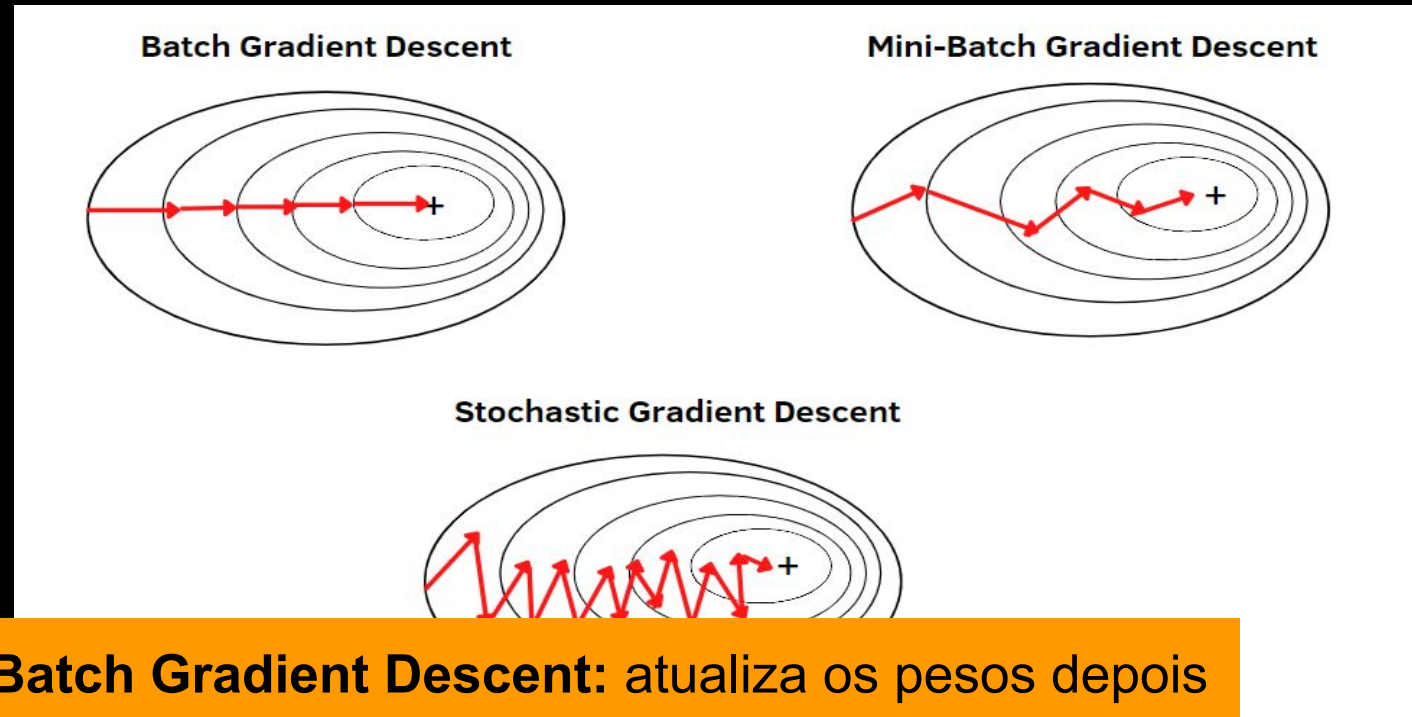
(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)





Gradient Descent ou Batch Gradient Descent: atualiza os pesos depois de ver o conjunto completo de exemplos. A direção do gradiente é mais precisa.

Stochastic Gradient Descent atualiza os pesos após ver cada exemplo. Assim a direção do gradiente é ruidosa.

Mini-Batch Gradient Descent: atualiza os pesos depois de ver um subconjunto (mini-batch) de exemplos.

Stochastic gradient descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Considerando que temos m exemplos

```
Loop {  
    for i=1 to m, {  
         $\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$  (for every  $j$ )  
    }  
}
```

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

Stochastic gradient descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Considerando que temos m exemplos

```
Loop {  
    for i=1 to m, {  
         $\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$  (for every  $j$ )  
    }  
}
```

atualiza os pesos após ver
cada exemplo de treinamento

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

Stochastic gradient descent

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Considerando que temos m exemplos

Loop {

for i=1 to m, {

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)} \quad (\text{for every } j)$$

}

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- Quando o conjunto de treinamento é grande, o stochastic gradient descent é preferido quando comparado com o batch gradient descent.
- Stochastic gradient descent chega a um J “perto” do mínimo mais rápido do que o batch gradient descent

$$= \theta^T x$$

Escolhendo α

Para saber se o gradiente descendente está funcionando, imprimir a função de custo J em cada iteração

- O valor deve diminuir em cada iteração
- Caso não, ajustar α

INTELIGÊNCIA ARTIFICIAL

Regressão Linear

Karina Valdivia Delgado

Slides baseados nas notas do prof. Andrew Ng